

PCS-302 : Data Structures Lab

Singly Linked List | Create, InsertFront, InsertEnd, InsertAnywhere, DeleteEnd, DeleteAnywhere, Size, IsEmpty, FindMiddle

Problem Statement

Write an algorithm and a program to implement a singly linked list. The program should implement the following operations: Create(k), InsertFront(k), InsertEnd(k), InsertAnywhere(k, p), DeleteEnd(), DeleteAnywhere(p), Size(), IsEmpty(), FindMiddle().

Example

```
create(10)          --> List: 10
insertEnd(20)       --> List: 10 20
insertEnd(30)       --> List: 10 20 30
insertFront(5)      --> List: 5 10 20 30

insertAnywhere(15,3) --> List: 5 10 15 20 30   (inserted at position 3)

size()              --> 5
findMiddle()        --> 15

deleteEnd()          --> removes 30, List: 5 10 15 20
deleteAnywhere(2)    --> removes 10, List: 5 15 20
```

Algorithm

```
Create(k):
    head = NULL
    create new node with data = k
    head = new node

IsEmpty():
    return (head == NULL)

Size():
    count = 0, temp = head
    while temp != NULL:
        count = count + 1
        temp = temp->next
    return count

InsertFront(k):
    create new node with data = k
    newNode->next = head
    head = newNode

InsertEnd(k):
    create new node with data = k, next = NULL
    if list empty: head = newNode
    else: traverse to last node, set last->next = newNode

InsertAnywhere(k, p):
```

```

    if p <= 1 or list empty: InsertFront(k), return
    traverse to node at position (p-1)
    create new node with data = k
    newNode->next = temp->next
    temp->next = newNode

```

```

DeleteEnd():
    if list empty: return
    if only one node: remove head, head = NULL
    else: traverse to second-last node
           remove last node, second-last->next = NULL

```

```

DeleteAnywhere(p):
    if list empty: return
    if p <= 1: remove head, head = head->next
    else: traverse to node at position (p-1)
           remove temp->next
           temp->next = temp->next->next

```

```

FindMiddle():
    slow = head, fast = head
    while fast != NULL and fast->next != NULL:
        slow = slow->next
        fast = fast->next->next
    return slow->data          // slow is now at the middle

```

C Program

```

#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *head = NULL;

void create(int k) {
    head = NULL;
    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    newNode->data = k;
    newNode->next = NULL;
    head = newNode;
    printf("List created with first element %d.\n", k);
}

int isEmpty() {
    if (head == NULL)
        return 1;
    else
        return 0;
}

int size() {

```

```

    int count = 0;
    struct Node *temp = head;
    while (temp != NULL) {
        count++;
        temp = temp->next;
    }
    return count;
}

void insertFront(int k) {
    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    newNode->data = k;
    newNode->next = head;
    head = newNode;
    printf("%d inserted at front.\n", k);
}

void insertEnd(int k) {
    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    newNode->data = k;
    newNode->next = NULL;

    if (isEmpty() == 1) {
        head = newNode;
    } else {
        struct Node *temp = head;
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = newNode;
    }
    printf("%d inserted at end.\n", k);
}

void insertAnywhere(int k, int p) {
    // p is the position (1-based) where k should be inserted
    if (p <= 1 || isEmpty() == 1) {
        insertFront(k);
        return;
    }

    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    newNode->data = k;

    struct Node *temp = head;
    int i;
    for (i = 1; i < p - 1 && temp->next != NULL; i++) {
        temp = temp->next;
    }

    newNode->next = temp->next;
    temp->next = newNode;
    printf("%d inserted at position %d.\n", k, p);
}

void deleteEnd() {

```

```

    if (isEmpty() == 1) {
        printf("List is empty. Cannot delete.\n");
        return;
    }

    if (head->next == NULL) {
        printf("%d deleted from end.\n", head->data);
        free(head);
        head = NULL;
        return;
    }

    struct Node *temp = head;
    while (temp->next->next != NULL) {
        temp = temp->next;
    }
    printf("%d deleted from end.\n", temp->next->data);
    free(temp->next);
    temp->next = NULL;
}

void deleteAnywhere(int p) {
    if (isEmpty() == 1) {
        printf("List is empty. Cannot delete.\n");
        return;
    }

    if (p <= 1) {
        struct Node *toDelete = head;
        printf("%d deleted from position %d.\n", head->data, p);
        head = head->next;
        free(toDelete);
        return;
    }

    struct Node *temp = head;
    int i;
    for (i = 1; i < p - 1 && temp->next != NULL; i++) {
        temp = temp->next;
    }

    if (temp->next == NULL) {
        printf("Position %d does not exist.\n", p);
        return;
    }

    struct Node *toDelete = temp->next;
    printf("%d deleted from position %d.\n", toDelete->data, p);
    temp->next = toDelete->next;
    free(toDelete);
}

void findMiddle() {
    if (isEmpty() == 1) {
        printf("List is empty.\n");
    }
}

```

```

        return;
    }

    struct Node *slow = head;
    struct Node *fast = head;

    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
    }

    printf("Middle element = %d\n", slow->data);
}

void printList() {
    struct Node *temp = head;
    if (isEmpty() == 1) {
        printf("List is empty.\n");
        return;
    }
    printf("List: ");
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

int main() {
    create(10);
    insertEnd(20);
    insertEnd(30);
    insertFront(5);
    printList();

    insertAnywhere(15, 3);
    printList();

    printf("Size = %d\n", size());

    findMiddle();

    deleteEnd();
    printList();

    deleteAnywhere(2);
    printList();

    if (isEmpty() == 1)
        printf("List is empty.\n");
    else
        printf("List is not empty.\n");

    return 0;
}

```

